

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
VIỆN TRÍ TUỆ NHÂN TẠO**



**BÁO CÁO MÔN HỌC KỸ THUẬT VÀ CÔNG
NGHỆ DỮ LIỆU LỚN CHO TRÍ TUỆ NHÂN TẠO**

ĐỀ TÀI

**AI/LLM ADOPTION TRACKER:
REAL-TIME GITHUB EVENTS
PROCESSING**

Nhóm sinh viên thực hiện (Nhóm 16):

1. Lê Thị Khánh Linh - 24022384
2. Trần Bùi Hà Giang - 24022314

Giảng viên hướng dẫn:

TS. Trần Hồng Việt
ThS. Ngô Minh Hương

HÀ NỘI - 2026

Mục lục

MỞ ĐẦU	3
1 TỔNG QUAN DỰ ÁN	4
1.1 Bối cảnh thực hiện	4
1.2 Mục tiêu dự án	4
1.3 Phạm vi và Giới hạn	4
2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ÁP DỤNG	6
2.1 Apache Kafka trong vai trò Message Broker	6
2.1.1 Cơ chế hoạt động	6
2.1.2 Vai trò trong hệ thống	6
2.2 Apache Spark và module Structured Streaming	6
2.2.1 Cơ chế hoạt động	6
2.2.2 Vai trò trong hệ thống	7
2.3 Cơ sở dữ liệu in-memory Redis	7
2.3.1 Vai trò trong hệ thống	7
2.4 Thành phần trực quan hóa và Quản lý hạ tầng	7
3 THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG	9
3.1 Kiến trúc luồng dữ liệu	9
3.2 Triển khai tầng thu thập dữ liệu	9
3.3 Triển khai tầng xử lý luồng	10
3.4 Thiết kế cấu trúc dữ liệu lưu trữ	10
4 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	12
4.1 Môi trường và Kết quả thu thập	12
4.2 Đánh giá khả năng duy trì trạng thái (Fault Tolerance)	13
5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	15
5.1 Kết luận	15
5.2 Hướng mở rộng	15
PHÂN CÔNG CÔNG VIỆC	16
TÀI LIỆU THAM KHẢO	17

Danh sách hình vẽ

1	Sơ đồ kiến trúc luồng dữ liệu hệ thống	9
2	Giao diện Bảng điều khiển (Dashboard) tại thời điểm kết thúc thực nghiệm	12
3	Bảng thống kê số lượt nhắc đến theo từ khóa công nghệ	13
4	Bản ghi log hệ thống trong thời điểm xảy ra ngoại lệ kết nối mạng	14
5	Bản ghi log quá trình khởi tạo lại tiến trình đọc từ Checkpoint của Spark .	14

Danh sách bảng

1	Phân công nhiệm vụ chi tiết	16
---	---------------------------------------	----

MỞ ĐẦU

Trong lĩnh vực kỹ thuật phần mềm hiện nay, các dự án mã nguồn mở trên GitHub tạo ra một lượng lớn dữ liệu phản ánh xu hướng phát triển và mức độ quan tâm của cộng đồng lập trình viên đối với các công nghệ mới. Đặc biệt, sự phát triển mạnh mẽ của các công nghệ liên quan đến Trí tuệ nhân tạo (AI) và Mô hình ngôn ngữ lớn (LLM), như RAG, LangChain hay các dịch vụ của OpenAI, đặt ra nhu cầu thu thập và phân tích dữ liệu theo thời gian thực nhằm theo dõi mức độ phổ biến của các công nghệ này.

Trong khi đó, các phương pháp phân tích dựa trên việc truy vấn dữ liệu định kỳ thường gặp hạn chế về khả năng cập nhật và xử lý liên tục khi khối lượng dữ liệu phát sinh ngày càng lớn. Điều này làm nổi bật vai trò của các hệ thống xử lý dữ liệu luồng (stream processing) trong việc xây dựng các nền tảng phân tích hiện đại.

Dự án “AI/LLM Adoption Tracker” được thực hiện nhằm xây dựng một hệ thống phân tán có khả năng tiếp nhận, xử lý và trực quan hóa các sự kiện từ GitHub thông qua một quy trình xử lý dữ liệu (Data Pipeline). Hệ thống được triển khai theo mô hình micro-batch, kết hợp các công nghệ xử lý dữ liệu quy mô lớn như Apache Kafka, Apache Spark Structured Streaming và Redis để bảo đảm khả năng mở rộng cũng như hiệu năng xử lý.

Báo cáo tập trung trình bày kiến trúc tổng thể của hệ thống, cơ chế tách biệt giữa quá trình thu thập và xử lý dữ liệu, phương pháp tính toán trên các cửa sổ thời gian (windowing), đồng thời phân tích các giải pháp bảo đảm tính liên tục của dữ liệu và khả năng phục hồi khi xảy ra sự cố trong quá trình vận hành hệ thống.

Mã nguồn và Video Demo dự án:

- **GitHub Repository:** [khanhlinh1308/Nhom16-AI-LLM-Adoption-Tracker](#)
- **Video Demo:** [Google Drive - Video Demo Sản Phẩm](#)

1 TỔNG QUAN DỰ ÁN

1.1 Bối cảnh thực hiện

Tần suất tương tác đối với các công nghệ AI thay đổi liên tục. Sự chuyển dịch này được phản ánh trực tiếp qua các hành vi trên GitHub như đẩy mã nguồn mới, mở yêu cầu gộp mã hay khởi tạo kho lưu trữ. Khối lượng sự kiện sinh ra từ GitHub Events API mang đặc tính của dữ liệu luồng (streaming data): quy mô biến động theo từng thời điểm, cấu trúc JSON đa dạng và phát sinh liên tục. Việc đo lường xu hướng đòi hỏi một hệ thống có khả năng thu thập và tổng hợp thông tin với độ trễ tính bằng giây, thay vì phụ thuộc vào các tập dữ liệu lịch sử được trích xuất thủ công theo từng đợt (batch processing).

1.2 Mục tiêu dự án

Dự án tập trung vào việc thiết kế và lập trình một luồng dữ liệu (Data Pipeline) từ khâu thu thập đến trực quan hóa. Các mục tiêu kỹ thuật cụ thể bao gồm:

- Lập trình tiến trình truy xuất dữ kiện từ GitHub Events API, tích hợp cơ chế quản lý giới hạn tần suất gọi hàm (Rate Limit).
- Xây dựng cơ chế hàng đợi thông điệp để lưu trữ tạm thời các sự kiện thô, đảm bảo tính chịu lỗi và tránh mất mát dữ liệu khi tải lượng hệ thống tăng đột biến.
- Thiết lập engine xử lý luồng để làm sạch dữ liệu, đối sánh chuỗi và tính toán tần suất xuất hiện của các từ khóa AI theo cửa sổ thời gian.
- Phát triển bảng điều khiển trực quan (Dashboard) truy vấn dữ liệu từ cơ sở dữ liệu in-memory nhằm hiển thị các chỉ số tổng hợp theo thời gian thực, bao gồm tổng số sự kiện phát hiện được, phân bố công nghệ và danh sách các kho lưu trữ có nhiều tương tác nhất.

1.3 Phạm vi và Giới hạn

- **Nguồn dữ liệu:** Khai thác luồng sự kiện công khai từ GitHub Events API, giới hạn trong bốn loại sự kiện chứa thông tin văn bản có giá trị phân tích bao gồm `PushEvent`, `PullRequestEvent`, `CreateEvent` và `WatchEvent`.
- **Phương pháp xử lý logic:** Phân tích dựa trên kỹ thuật đối sánh chuỗi (string matching) giữa nội dung sự kiện và một từ điển từ khóa thiết lập sẵn. Hệ thống không tích hợp các mô hình học máy hay kỹ thuật xử lý ngôn ngữ tự nhiên (NLP) để phân loại ngữ cảnh.

- **Cơ chế lưu trữ:** Hệ thống tính toán theo phương pháp cuốn chiếu trên các cửa sổ thời gian (Windowing). Kết quả tổng hợp được lưu tại bộ nhớ in-memory để tối ưu hóa tốc độ hiển thị trên giao diện. Dự án không thiết lập hệ thống lưu trữ thứ cấp hoặc cơ sở dữ liệu quan hệ truyền thống để lưu trữ dài hạn (Cold Storage) đối với luồng dữ liệu thô.

2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ÁP DỤNG

2.1 Apache Kafka trong vai trò Message Broker

Apache Kafka là một nền tảng phân phối luồng sự kiện phân tán, đóng vai trò trung gian gắn kết giữa nguồn cấp dữ liệu GitHub API và hệ thống xử lý luồng Apache Spark [2].

2.1.1 Cơ chế hoạt động

Kafka tổ chức dữ liệu theo các chủ đề và chia nhỏ thành các phân vùng. Tiến trình gửi dữ liệu (Producer) ghi các thông điệp vào cuối phân vùng, trong khi tiến trình nhận dữ liệu (Consumer) đọc dữ liệu tuần tự dựa trên một con trỏ định vị gọi là *offset*. Dữ liệu được lưu trữ trên đĩa cứng của Kafka broker trong một khoảng thời gian cấu hình trước, cho phép Consumer có thể truy xuất và đọc lại dữ liệu khi cần thiết [2].

2.1.2 Vai trò trong hệ thống

Tốc độ sinh dữ liệu từ GitHub biến động không đồng đều, thường xuyên xảy ra tình trạng bùng nổ dữ liệu với số lượng lớn trong một khoảng thời gian ngắn (data burst). Nếu tiến trình thu thập đẩy dữ liệu trực tiếp vào tiến trình xử lý, sự chênh lệch tốc độ có thể gây ra hiện tượng thắt cổ chai (bottleneck) hoặc tràn bộ nhớ hệ thống.

Kafka được cấu hình để tách biệt hai tiến trình này bằng cách cung cấp một vùng đệm an toàn trên đĩa cứng. Tiến trình thu thập chỉ thực hiện ghi dữ liệu thô vào Kafka. Đồng thời, engine xử lý của Spark sẽ chủ động kéo dữ liệu từ Kafka với tốc độ phù hợp với năng lực cấp phát tài nguyên tính toán của cụm. Cơ chế quản lý offset của Kafka đóng vai trò cốt lõi trong việc đảm bảo hệ thống nhận biết chính xác vị trí tiến trình đọc đã dừng lại khi xảy ra sự cố.

2.2 Apache Spark và module Structured Streaming

Apache Spark là một framework tính toán phân tán hiệu năng cao. Module Spark Structured Streaming thực thi việc xử lý luồng dựa trên mô hình micro-batch, trong đó luồng dữ liệu liên tục được chia thành các lô nhỏ để xử lý theo chu kỳ thời gian định sẵn[1].

2.2.1 Cơ chế hoạt động

Structured Streaming coi luồng dữ liệu đầu vào như một bảng vô hạn được bổ sung dữ liệu liên tục. Nhà phát triển định nghĩa các phép biến đổi dữ liệu trên bảng này bằng các API của DataFrame hoặc SQL. Engine của Spark sẽ tự động lập lịch để chạy các truy vấn gia tăng (incremental queries) trên phần dữ liệu mới nhận được, sau đó cập nhật kết quả đầu ra theo chu kỳ thời gian được cấu hình bởi bộ trigger [1].

2.2.2 Vai trò trong hệ thống

Xử lý dữ liệu từ GitHub đòi hỏi các phép biến đổi cấu trúc JSON, làm sạch chuỗi ký tự và gom nhóm dữ liệu. Thay vì duy trì trạng thái của toàn bộ lịch sử dữ liệu trong bộ nhớ chính, Spark Structured Streaming được định nghĩa để tính toán trên các cửa sổ thời gian (Windowing). Hệ thống chỉ duy trì kết quả tổng hợp trong bộ nhớ và cập nhật liên tục theo từng lô nhỏ micro-batch.

Đặc biệt, Spark cung cấp cơ chế Checkpointing tự động ghi lại trạng thái tính toán hiện tại và offset của Kafka vào đĩa cứng thông qua cơ chế Write-Ahead Logs. Đặc điểm này cho phép hệ thống khôi phục trạng thái bộ nhớ và xác định chính xác vị trí cần đọc lại dữ liệu trên Kafka nếu tiến trình bị dừng đột ngột, đáp ứng hoàn hảo yêu cầu chịu lỗi (Fault Tolerance) của dự án.

2.3 Cơ sở dữ liệu in-memory Redis

Redis là hệ quản trị cơ sở dữ liệu lưu trữ dữ liệu trực tiếp trên bộ nhớ chính (RAM), mang lại tốc độ phản hồi tối ưu so với các cơ sở dữ liệu quan hệ truyền thống ghi xuống ổ cứng [3].

2.3.1 Vai trò trong hệ thống

Yêu cầu kỹ thuật của bảng điều khiển trực quan là tự động làm mới và vẽ lại biểu đồ dựa trên dữ liệu mới nhất. Việc thực thi các câu lệnh `GROUP BY` và `ORDER BY` liên tục trên cơ sở dữ liệu quan hệ sẽ tạo ra độ trễ I/O lớn, ảnh hưởng đến hiệu năng truy xuất thời gian thực.

Sử dụng Redis giúp giải quyết triệt để vấn đề độ trễ truy xuất bằng cách khai thác các cấu trúc dữ liệu đặc thù:

- **Sorted Set (ZSET):** Cấu trúc dữ liệu này gán một điểm số cho mỗi phần tử và tự động sắp xếp theo thứ tự. Thuật toán chèn và cập nhật của ZSET có độ phức tạp thời gian là $O(\log(N))$. Hệ thống sử dụng ZSET để lưu trữ và liên tục cập nhật bảng xếp hạng (leaderboard) cho các từ khóa công nghệ và tên kho lưu trữ.
- **List:** Cấu trúc danh sách liên kết hỗ trợ thao tác chèn vào đầu chuỗi với độ phức tạp thời gian tối ưu $O(1)$ [3], được sử dụng để lưu trữ chuỗi giá trị tổng số sự kiện theo trình tự thời gian cho biểu đồ đường.

2.4 Thành phần trực quan hóa và Quản lý hạ tầng

- **Streamlit:** Là thư viện Python được sử dụng để xây dựng giao diện hiển thị. Streamlit cho phép trích xuất cấu trúc dữ liệu từ Redis và kết xuất trực tiếp thành

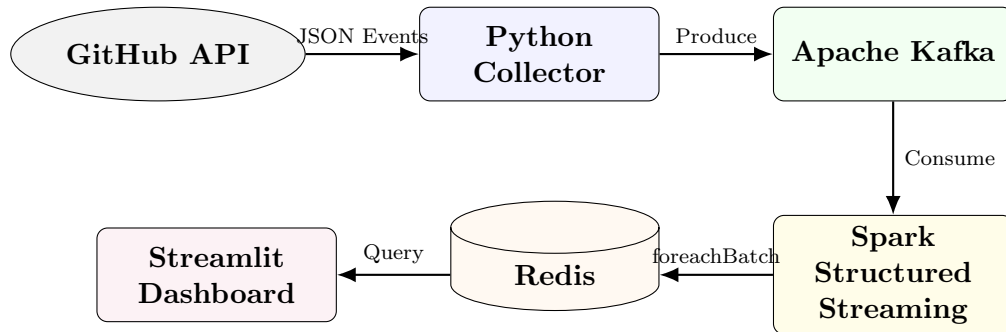
các biểu đồ cột, biểu đồ đường và bảng số liệu mà không cần thiết lập hạ tầng Backend và Frontend tách biệt [4].

- **Docker Compose:** Các thành phần phân tán của hệ thống bao gồm Zookeeper, Kafka, Spark Master, Spark Worker, Redis và Streamlit đòi hỏi các môi trường thực thi khác nhau. Docker Compose được sử dụng để định nghĩa cấu hình và đóng gói toàn bộ các dịch vụ này thành các container độc lập. Việc này đảm bảo tính đồng nhất khi khởi chạy hệ thống trên các máy chủ vật lý khác nhau, đồng thời hỗ trợ quản lý định tuyến mạng nội bộ (network routing) giữa các dịch vụ.

3 THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG

3.1 Kiến trúc luồng dữ liệu

Hệ thống được thiết kế theo hướng trực tiếp, chuyển tiếp dữ liệu qua bốn tầng chức năng chính. Mỗi tầng thực hiện một tác vụ chuyên biệt, giảm thiểu sự phụ thuộc trực tiếp lẫn nhau.



Hình 1: Sơ đồ kiến trúc luồng dữ liệu hệ thống

3.2 Triển khai tầng thu thập dữ liệu

Tầng thu thập được vận hành thông qua một tiến trình Python độc lập (`github_collector.py`). Tiến trình này thực thi vòng lặp vô hạn, gọi HTTP GET đến điểm cuối sự kiện của GitHub theo chu kỳ cấu hình.

Để kiểm soát giới hạn tài nguyên của giao diện lập trình ứng dụng, tiến trình tích hợp mã thông báo xác thực cá nhân vào header, đồng thời kiểm tra giá trị trả về `X-RateLimit-Remaining`. Hệ thống chỉ lọc các sự kiện thuộc danh sách định nghĩa: `PushEvent`, `CreateEvent`, `PullRequestEvent` và `WatchEvent` [5].

Dữ liệu thô thu về được trích xuất thành cấu trúc từ điển chứa các trường cơ bản: `event_id`, `event_type`, `repo_name`, `actor_login`, `created_at` và `text_content` (tổng hợp từ tên kho lưu trữ và chi tiết nội dung sự kiện). Đối tượng này sau đó được tuần tự hóa thành chuỗi JSON và chuyển cho Kafka Producer để đẩy vào chủ đề `github_events`.

```
1 {
2   "event_id": "sample-001",
3   "event_type": "PushEvent",
4   "repo_name": "team16/langchain-rag-demo",
5   "actor_login": "demo-user",
6   "created_at": "2026-06-19T03:00:00Z",
7   "text_content": "Add LangChain RAG pipeline with OpenAI embeddings"
8 }
```

Listing 1: Lược đồ cấu trúc dữ liệu JSON sinh ra từ GitHub Events

3.3 Triển khai tầng xử lý luồng

Logic xử lý được định nghĩa bằng PySpark, kết nối với cụm Spark Master thông qua Docker. Quá trình xử lý một lô nhỏ dữ liệu (micro-batch) bao gồm các pha sau:

Pha 1: Trích xuất và làm sạch dữ liệu

Spark đọc luồng byte từ Kafka và giải mã bằng hàm `from_json` dựa trên lược đồ dữ liệu đã định nghĩa. Các trường văn bản được chuyển thành chữ thường. Biểu thức chính quy `regexp_replace` được áp dụng để loại bỏ các ký tự đặc biệt, dấu câu không cần thiết nhằm hỗ trợ quá trình đối sánh chuỗi. Chuỗi `created_at` được ép sang kiểu dữ liệu thời gian chuẩn.

Pha 2: Đối sánh từ khóa

Hệ thống duy trì một danh sách các từ khóa công nghệ AI như *chatgpt*, *llm*, *rag*, *claude*, *openai*, *ai agent*,... Hàm điều kiện `when().contains()` được sử dụng để duyệt qua trường văn bản tổng hợp. Nếu có sự trùng khớp, sự kiện được gắn thêm cột chứa mảng các từ khóa phát hiện được. Lệnh `explode` được gọi để tách mảng này thành các dòng riêng biệt, phục vụ việc thống kê độc lập cho từng công nghệ.

Pha 3: Tính toán trên cửa sổ thời gian

Sử dụng hàm `window`, hệ thống nhóm các sự kiện theo khoảng thời gian thực tế mà sự kiện phát sinh. Phép toán `count` và `collect_list` được thực thi để đếm số lượt xuất hiện của từ khóa và danh sách các kho lưu trữ tương ứng trong cửa sổ thời gian đó. Spark quản lý tình trạng dữ liệu đến trễ bằng khái niệm Watermark, loại bỏ các sự kiện vượt quá ngưỡng thiết lập.

Pha 4: Cập nhật cơ sở dữ liệu

Do Spark không hỗ trợ cơ chế ghi luồng trực tiếp vào Redis, dự án sử dụng phương thức `foreachBatch`. Phương thức này nhận đầu vào là một `DataFrame` đại diện cho kết quả của một micro-batch. Một hàm Python cục bộ được gọi để thu thập dữ liệu về dạng bản ghi và sử dụng thư viện `redis-py` để thực thi lệnh Pipeline ghi vào bộ nhớ in-memory với chu kỳ kích hoạt (trigger) định tuyến mỗi 30 giây.

3.4 Thiết kế cấu trúc dữ liệu lưu trữ

Cơ sở dữ liệu Redis được cấp phát các khóa với định dạng cấu trúc tối ưu như sau:

- Cấu trúc ZSET `top_ai_keywords`: Lưu trữ danh sách các từ khóa AI. Điểm số được tự động cộng dồn và cập nhật thông qua lệnh `ZINCRBY`.
- Cấu trúc ZSET `top_ai_repos`: Lưu trữ tên các kho lưu trữ, có điểm số được cập nhật tương tự bằng lệnh `ZINCRBY`.
- Cấu trúc String `total_ai_events_detected`: Biến đếm tổng số sự kiện AI phát hiện được. Giá trị được cộng dồn thông qua lệnh `INCRBY`.

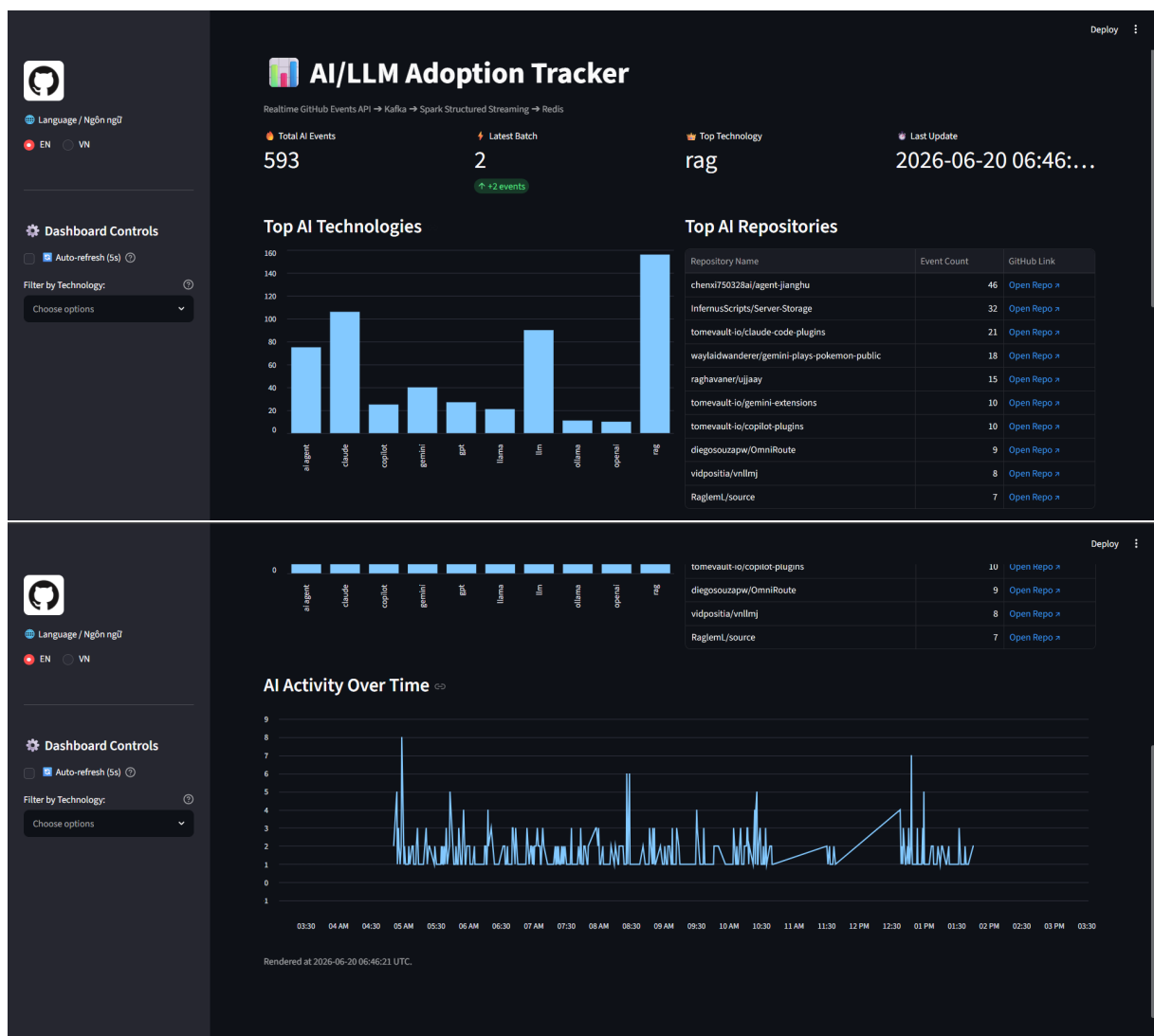
- Cấu trúc List `ai_activity_timeline`: Lưu thông tin tổng quan của mỗi batch dưới dạng JSON. Thao tác chèn dữ liệu mới thực hiện bằng `LPUSH`. Để kiểm soát dung lượng bộ nhớ, lệnh `LTRIM` được áp dụng nhằm duy trì đúng 1000 phần tử mới nhất.

4 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Môi trường và Kết quả thu thập

Hệ thống được thiết lập và duy trì hoạt động trong điều kiện thực nghiệm kéo dài khoảng 9 giờ (từ 04 giờ 50 phút đến 13 giờ 47 phút). Trong suốt quá trình này, các container duy trì trạng thái hoạt động ổn định mà không ghi nhận lỗi tràn bộ nhớ hoặc dừng tiến trình đột ngột.

Giao diện Streamlit thực hiện truy vấn định kỳ vào bộ nhớ Redis và kết xuất thành biểu đồ trực quan. Tại thời điểm dừng thực nghiệm, dữ liệu hiển thị tổng cộng **593 sự kiện AI** đã được định danh và lưu trữ.



Hình 2: Giao diện Bảng điều khiển (Dashboard) tại thời điểm kết thúc thực nghiệm

Giao diện cung cấp chức năng đa ngôn ngữ và cơ chế làm mới dữ liệu tự động dựa trên đồng hồ đếm ngược. Biểu đồ đường mô phỏng biến động khối lượng sự kiện theo thời gian thực, cho thấy dữ liệu được cập nhật thành các đỉnh dữ liệu (spike) tương ứng

với chu kỳ thu thập.

Dựa trên cấu trúc ZSET của Redis, bảng phân tích top công nghệ ghi nhận số liệu tần suất xuất hiện như sau:

Top AI Technologies		
	keyword	count
0	rag	156
1	claude	106
2	llm	90
3	ai agent	75
4	gemini	40
5	gpt	27
6	copilot	25
7	llama	21
8	ollama	11

Hình 3: Bảng thống kê số lượt nhắc đến theo từ khóa công nghệ

Từ khóa **rag** ghi nhận khối lượng tương tác lớn nhất với 156 lượt nhắc đến. Theo sau là **claude** (106 lượt), **llm** (90 lượt) và **ai agent** (75 lượt). Mặc dù bảng dữ liệu trích xuất bị khuất một phần hiển thị, nhưng đối chiếu trên biểu đồ tổng thể, các công cụ mô hình ngôn ngữ khác như **gemini** (40), **gpt** (27), **copilot** (25), **llama** (21), **ollama** (11) và **openai** (10) cũng được ghi nhận đầy đủ.

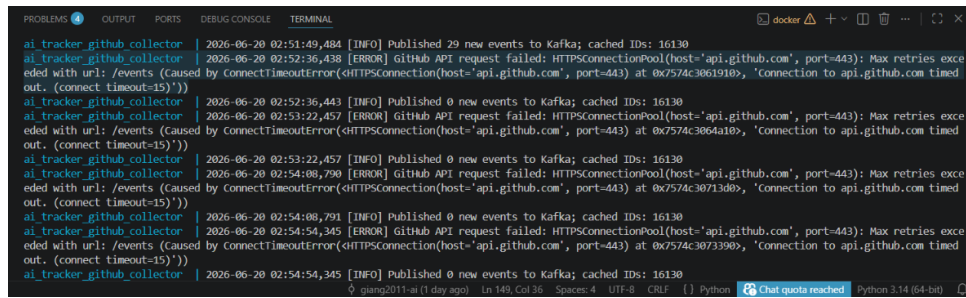
Ở khía cạnh kho lưu trữ, repository `chenxi750328ai/agent-jianghu` đứng đầu danh sách tương tác với 46 sự kiện được thu thập.

4.2 Đánh giá khả năng duy trì trạng thái (Fault Tolerance)

Việc phân tách kiến trúc thành các dịch vụ độc lập thông qua Message Broker và cơ chế lưu trạng thái của Spark cho thấy hiệu quả rõ rệt trong việc xử lý sự cố. Quá trình thực nghiệm đã phát sinh hai sự kiện lỗi kỹ thuật:

Kịch bản 1: Gián đoạn kết nối từ nguồn cấp dữ liệu

Khoảng 09 giờ 52 phút, yêu cầu HTTP gửi đến điểm cuối API của GitHub không nhận được phản hồi, hệ thống sinh ra ngoại lệ `Connection timed out`. Thông thường, ngoại lệ này sẽ chấm dứt hoàn toàn tiến trình chạy nền. Tuy nhiên, khối lệnh bắt lỗi và vòng lặp thử lại (retry loop) bên trong `github_collector.py` đã tiếp quản ngoại lệ một cách an toàn. Tiến trình đi vào trạng thái chờ và thực hiện kết nối lại thành công sau đó, tiếp tục ghi dữ liệu vào Kafka. Kiến trúc này giới hạn phạm vi ảnh hưởng của lỗi mạng chỉ ở tầng thu thập, không làm gián đoạn tầng thông điệp Kafka hay tầng xử lý Spark.

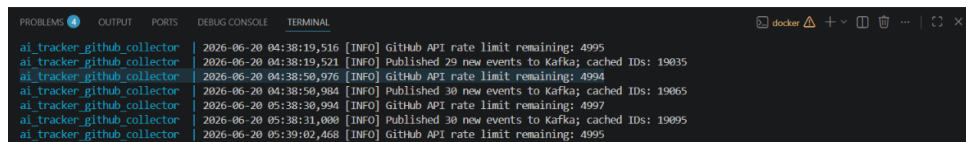


Hình 4: Bản ghi log hệ thống trong thời điểm xảy ra ngoại lệ kết nối mạng

Kịch bản 2: Gián đoạn tài nguyên máy chủ

Trong quá trình thực nghiệm, máy chủ vật lý rơi vào trạng thái ngủ đông (Sleep) hơn 1 giờ đồng hồ. Hệ quả là tài nguyên CPU và RAM bị ngắt cấp phát cho Docker, luồng xử lý của Spark buộc phải dừng lại. Khác với kiến trúc truyền thông trực tiếp qua giao thức socket (dữ liệu đang truyền tải sẽ bị hủy bỏ khi đứt kết nối), sự kết hợp giữa Kafka và Spark giải quyết triệt để vấn đề này bằng nguyên lý kiểm tra điểm checkpoint.

Spark ghi nhận liên tục giá trị *offset* của thông điệp cuối cùng đọc được từ Kafka vào thư mục kiểm tra điểm `/tmp/spark_checkpoints`. Khi máy chủ khôi phục tài nguyên, Spark truy vấn lại thư mục này, xác định vị trí *offset* đã lưu và yêu cầu Kafka cấp phát dữ liệu tiếp nối từ đúng vị trí đó. Tiến trình nhanh chóng tiêu thụ lượng sự kiện bị dồn ứ (backlog) trong suốt thời gian máy ngủ đông. Nhờ cơ chế quản lý trạng thái khép kín này, hệ thống không để xảy ra hiện tượng mất mát sự kiện hay tính toán trùng lặp.



Hình 5: Bản ghi log quá trình khởi tạo lại tiến trình đọc từ Checkpoint của Spark

5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Dự án đã thiết kế và triển khai thành công một luồng dữ liệu (Data Pipeline) toàn diện, đảm nhiệm chu trình khép kín từ thu thập, xử lý đến trực quan hóa các sự kiện GitHub liên quan đến công nghệ AI và LLM.

Hệ thống được kiến trúc từ các công nghệ tiêu chuẩn công nghiệp bao gồm: tiến trình thu thập dữ liệu bằng Python, hệ thống hàng đợi thông điệp Apache Kafka, engine xử lý luồng Apache Spark Structured Streaming, cơ sở dữ liệu in-memory Redis và bảng điều khiển trực quan Streamlit.

Dự án đã giải quyết bài toán theo dõi xu hướng công nghệ bằng cách tính toán trực tiếp trên dữ liệu phát sinh theo thời gian thực, thay vì phụ thuộc vào các tập dữ liệu lưu trữ tĩnh. Kết quả thực nghiệm suốt gần 9 giờ đồng hồ chứng minh hệ thống vận hành ổn định dưới áp lực sự kiện liên tục. Đặc biệt, kiến trúc phân tách qua Message Broker kết hợp cùng cơ chế Checkpointing đã thể hiện khả năng chịu lỗi (fault tolerance) xuất sắc, giúp pipeline tự động khôi phục hoàn toàn trạng thái và dữ liệu sau các sự cố đứt gãy kết nối hay gián đoạn tài nguyên phần cứng.

Tuy nhiên, giới hạn hiện tại của kiến trúc là chưa thiết lập kho lưu trữ dữ liệu thô vĩnh viễn, dẫn đến những hạn chế trong việc thực thi các phép phân tích hồi quy hay đối chiếu dữ liệu lịch sử trong dài hạn.

5.2 Hướng mở rộng

Dựa trên nền tảng kiến trúc đã xây dựng, hệ thống có thể tiếp tục được nâng cấp theo các hướng tiếp cận sau:

- **Xây dựng kho dữ liệu (Data Lake):** Bổ sung một luồng Consumer độc lập để kéo dữ liệu thô từ Kafka và ghi trực tiếp xuống hệ thống lưu trữ phân tán như HDFS hoặc cơ sở dữ liệu phi quan hệ như MongoDB. Bước tiến này sẽ giải quyết bài toán lưu trữ dài hạn, tạo tiền đề phục vụ các tiến trình phân tích theo lô (Batch Processing) hoặc huấn luyện mô hình máy học trong tương lai.
- **Tích hợp Xử lý ngôn ngữ tự nhiên (NLP):** Bổ sung mô-đun phân tích ngôn ngữ vào tiến trình Spark trước công đoạn thống kê tần suất. Việc ứng dụng các thuật toán Phân tích cảm xúc (Sentiment Analysis) để nhận diện sắc thái (tích cực, tiêu cực, trung tính) trong nội dung Commit hoặc Pull Request sẽ cung cấp thêm chiều không gian phân tích sâu hơn về chất lượng đóng góp mã nguồn, thay vì chỉ đo lường đơn thuần về mặt khối lượng sự kiện.

PHỤ LỤC: PHÂN CÔNG CÔNG VIỆC

Các hạng mục công việc trong dự án được phân bổ cho các thành viên Nhóm 16 như sau:

Họ và tên	Nhiệm vụ đảm nhận
Lê Thị Khánh Linh (Project Lead)	- Thiết kế kiến trúc và soạn thảo tài liệu đặc tả hệ thống. - Lập trình giao diện Dashboard bằng Streamlit. - Biên soạn Báo cáo chuyên đề và thiết kế Slide thuyết trình. - Thực hiện ghi hình và biên tập Video kiểm thử sản phẩm.
Trần Bùi Hà Giang (Core Data Engineer)	- Lập trình script thu thập dữ liệu từ GitHub API. - Thiết lập cấu trúc và khởi tạo Apache Kafka. - Lập trình thuật toán xử lý luồng, đối sánh từ khóa, ghi dữ liệu vào Redis bằng PySpark Streaming. - Thiết lập hạ tầng và triển khai hệ thống thông qua cấu hình Docker Compose.

Bảng 1: Phân công nhiệm vụ chi tiết

Tài liệu

- [1] Apache Spark. *Structured Streaming Programming Guide*. [Trực tuyến]. Khả dụng tại: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- [2] Apache Kafka. *Introduction to Kafka*. [Trực tuyến]. Khả dụng tại: <https://kafka.apache.org/documentation/>
- [3] Redis. *Redis Data Types*. [Trực tuyến]. Khả dụng tại: <https://redis.io/docs/data-types/>
- [4] Streamlit. *Streamlit Documentation*. [Trực tuyến]. Khả dụng tại: <https://docs.streamlit.io/>
- [5] GitHub. *GitHub Events API*. [Trực tuyến]. Khả dụng tại: <https://docs.github.com/en/rest/activity/events>